

Pandas Foundations

A Beginner's Lab Tutorial

Shared foundation for BCA-3007T Python Programming and BCA-3004T Feature Engineering

1. Why Are We Learning Pandas?

So far in your Python classes, you've worked with lists, dictionaries, and loops. That works fine for small amounts of data. But real-world data — student records, sales numbers, patient records — comes as tables: rows and columns, usually stored in a file.

Pandas is a Python library built specifically to work with this kind of tabular data — quickly, and without writing 20 lines of loops to do something Excel does in one click.

By the end of this tutorial, you'll be able to load a real data file into Python and start asking questions of it.

2. What Is a Package, a Module, and a Library?

Before touching pandas, let's clear up three words you'll hear constantly.

- **Module** — a single file of Python code that someone has already written, containing useful functions you can reuse instead of writing from scratch. Think of it as one tool.
- **Package** — a collection of related modules bundled together, like a toolbox holding many related tools.
- **Library** — used loosely to mean the same thing as a package: a ready-made collection of code you can plug into your own program.

You don't build a hammer every time you need to hit a nail — you reach into the toolbox. That's exactly what `import` does in Python: it reaches into a toolbox someone else built and hands you the tools.

Pandas, NumPy, and Matplotlib are all examples of libraries/packages you'll use constantly in both your courses this semester.

3. What Exactly Is Pandas?

In one line:

Pandas is a Python library that lets you store, view, and work with data in table form — rows and columns — the way you would in Excel, but using code.

Why does that matter?

- Excel struggles past a few hundred thousand rows. Pandas handles millions comfortably.
- Excel requires manual clicking. Pandas lets you repeat the same operation on a new file instantly by re-running your code.
- Pandas is the standard tool used before any machine learning or feature engineering work — which is exactly why both your courses depend on it.

4. What Is a CSV File?

CSV stands for Comma-Separated Values. It's the simplest possible way to store a table as plain text — every row is one line, and every column is separated by a comma.

Here's what a CSV file actually looks like if you open it in Notepad:

```
Name, Age, Marks, City
Amit, 20, 78, Ghaziabad
Priya, 21, 85, Noida
Rohan, 19, 64, Delhi
Sneha, 22, 91, Ghaziabad
Vikram, 20, 55, Noida
```

Open the same file in Excel, and it automatically arranges into a neat grid. Open it in pandas, and you get the same grid — but now you can search, filter, and calculate on it using code.

Same data. Three different views. Same underlying file.

5. Installing and Importing Pandas

If pandas isn't already installed on your lab machine:

```
pip install pandas
```

To use it in your code, you always start with:

```
import pandas as pd
```

Why "as pd"? This is not compulsory — you could write `import pandas` and use `pandas.read_csv(...)` everywhere. But `pd` is a universal shorthand every pandas user in the world follows. Every tutorial, every Stack Overflow answer, every textbook uses `pd`. Follow the convention so your code is instantly readable to others (and theirs to you).

6. What Is a DataFrame?

This is the single most important word in pandas.

A DataFrame is pandas' name for a table: data arranged in rows and columns.

Let's build one from scratch first — no file involved — just so the concept is crystal clear before we bring files into the picture.

```
import pandas as pd

data = {
    "Name": ["Amit", "Priya", "Rohan"],
    "Marks": [78, 85, 64]
}

df = pd.DataFrame(data)
print(df)
```

Output:

	<i>Name</i>	<i>Marks</i>
0	<i>Amit</i>	78
1	<i>Priya</i>	85
2	<i>Rohan</i>	64

Notice: we gave pandas a Python dictionary, and it turned it into a proper table — automatically adding row numbers (called the index) starting from 0.

This is a DataFrame. Everything else we do in pandas is really just operations on this one object.

7. Loading a Real CSV File

Now let's do what you'll actually do in every lab session from here on: load data from a file instead of typing it by hand.

Step 1: Create a file called `students.csv` in the same folder as your Python script, with this content:

```
Name,Age,Marks,City
Amit,20,78,Ghaziabad
Priya,21,85,Noida
Rohan,19,64,Delhi
Sneha,22,91,Ghaziabad
Vikram,20,55,Noida
Neha,21,72,Delhi
Karan,23,45,Ghaziabad
Anjali,20,88,Noida
```

Step 2: Load it:

```
import pandas as pd
```

```
df = pd.read_csv("students.csv")
print(df)
```

Output:

	Name	Age	Marks	City
0	Amit	20	78	Ghaziabad
1	Priya	21	85	Noida
2	Rohan	19	64	Delhi
3	Sneha	22	91	Ghaziabad
4	Vikram	20	55	Noida
5	Neha	21	72	Delhi
6	Karan	23	45	Ghaziabad
7	Anjali	20	88	Noida

That's it — one line, `pd.read_csv()`, and your entire file is now a DataFrame you can work with in code.

8. First Exploration Commands — Getting a Feel of the Data

Before doing anything else with a new dataset, always run these five commands. This is a habit you should build from day one.

`.head()` — shows the first 5 rows

```
df.head()
```

`.shape` — number of rows and columns

```
df.shape
```

```
(8, 4)
```

This means: 8 rows (8 students), 4 columns (4 pieces of information about each).

`.columns` — lists all column names

```
df.columns
```

```
Index(['Name', 'Age', 'Marks', 'City'], dtype='object')
```

`.info()` — technical summary of the table

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Name    8 non-null      object
1   Age     8 non-null      int64
2   Marks   8 non-null      int64
3   City    8 non-null      object
```

`.describe()` — quick statistics for numeric columns

```
df.describe()
```

	Age	Marks
count	8.000000	8.000000
mean	20.750000	72.250000
std	1.164965	16.184843
min	19.000000	45.000000
25%	20.000000	61.750000
50%	20.500000	75.000000
75%	21.250000	85.250000
max	23.000000	91.000000

9. Selecting Data — Rows and Columns

Selecting one column

```
df["Marks"]
```

0	78
1	85
2	64
3	91
4	55
5	72
6	45
7	88

Name: Marks, dtype: int64

Selecting more than one column

```
df[["Name", "Marks"]]
```

Selecting one row by position

```
df.iloc[0]    # the first row
```

Filtering rows based on a condition

```
df[df["Marks"] > 75]
```

	Name	Age	Marks	City
1	Priya	21	85	Noida
3	Sneha	22	91	Ghaziabad
7	Anjali	20	88	Noida

Read that line as a sentence: "give me all rows from df, where the Marks column is greater than 75."

10. Lab Task — Try This Yourself

Using the students.csv file above, write code to answer:

1. How many students are there in total, and how many columns of information do we have?
2. Show only the Name and City columns.
3. Show all students from Ghaziabad.
4. Show all students who scored more than 70 marks and are from Noida. (Hint: combine two conditions using `&`, and wrap each condition in brackets — `df[(condition1) & (condition2)]`.)
5. What is the average Marks of all students? (Hint: `.describe()` already showed you this — but try `df["Marks"].mean()` directly.)